

Automated CI/CD Pipeline with Preview Environments

Jonathan Värild

Jennifer Ha

October 2026

1 Introduction

We have developed a Continuous Integration and Continuous Deployment (CI/CD) pipeline that automates tasks related to code testing, integration and end-to-end testing, and project deployment. At the base of this project we have taken a recent project featuring a recruitment application for an amusement park. The application allows users to manage their user accounts, fill out a work application, and has a dedicated administration page for recruiters. With this, we have set up an extensive testing workflow that automatically creates a temporary preview deployment for pull requests and manages production deployments with pushes to the main branch. Automation is orchestrated by GitHub Actions, while Terraform deploys the infrastructure on a Virtual Private Server (VPS) with the help of HCP Terraform for remote state management.

2 Architecture

The implementation consists of a variety of different platforms and technologies that together make all this possible. GitHub manages code version history through Git and also hosts GitHub Actions, which makes it possible to run complex workflows based on different triggers. Docker is used to containerize the application and is combined with GitHub Container Registry (GHCR), which hosts all the application images that we build. Terraform is our Infrastructure as Code (IaC) framework, which is combined with HCP Terraform for managed Terraform state management in the cloud, and together makes it possible to manage deployments to our VPS. Additionally, the VPS runs Nginx bundled in an image called Nginx-Proxy alongside another image called Acme-Companion.

The deployed application is split into two different images consisting of the Next.js app build and the database plus its initial schema. Each deployment has its own Docker network, effectively isolating the database and only making it available to the application itself. Nginx-Proxy automatically detects each deployment via special environment variables and automatically generates Nginx configurations that expose the deployments via HTTPS. Acme-Companion integrates with Nginx-Proxy and enables TLS using automated Let's Encrypt certificates for all exposed deployments.

3 Continuous Integration and Testing

The project uses GitHub Actions to automate code quality checks and testing on pushes and pull requests. The CI workflow runs ESLint to detect potential bad practices in the code, Prettier to check formatting, TypeScript to verify TypeScript types, and Vitest to run unit tests. These checks help identify problems early and maintain code quality throughout development.

Unit tests from the original recruitment application are reused and integrated into the automated CI pipeline. They cover application functionality such as authentication, services, React components, and validation. Simple integration tests have been implemented and run against a temporary PostgreSQL container on pull requests. The workflow sets up the database and application and checks that they can communicate with each other correctly.

GitHub branch protection requires the relevant status checks to pass before a pull request can be merged. These include CI, integration tests, preview deployment, and end-to-end tests. In addition,

CodeQL performs an automated static analysis to identify potential security vulnerabilities in the source code. Together, these measures help ensure that changes are tested and reviewed before reaching the main branch.

4 Continuous Delivery and Preview Environments

The purpose of implementing a CD workflow is to ensure that every proposed change is tested in an environment similar to that of production before it is merged into the main branch. When a pull request (PR) is opened, updated, or reopened, the preview workflow builds two Docker images. One for the Next.js application and one for the PostgreSQL database. Both images are tagged with the PR head commit SHA, making each image traceable to the exact version of the source code from which it was built. Before the images are published, Trivy scans them for known vulnerabilities. Images that pass the configured security checks are pushed to GHCR, where they can be retrieved during deployment.

Terraform then creates or updates an isolated preview environment for the pull request. Each environment uses its own HCP Terraform workspace and has separate application and database containers, a private Docker network, and a database volume. The application is deployed to a PR-specific HTTPS address, e.g. `pr-9.devops.jompecloud.com`. The Nginx-Proxy application routes requests to the correct application and uses Let's Encrypt to generate certificates that enable TLS. All this allows PR preview deployments to run independently of production. A smoke test is run after the deployment to check whether the application is accessible. If the smoke test passes, Playwright runs end-to-end tests to verify that the deployed application works correctly. All required CI, integration, deployment, and end-to-end checks must be passed before the PR can be merged with the main branch.

When the pull request is closed, a different workflow runs which uses Terraform to remove the preview application deployment. It deletes all Docker resources and the corresponding HCP Terraform workspace. This automated cleanup prevents unused preview environments from consuming unnecessary resources. After a PR is merged, the workflow reuses its tested Docker images, following the "build once, promote the tested artifact" principle. Trivy scans the images before Terraform deploys them to production. If the main branch update cannot be associated with a merged pull request, new images are built and tested instead. Finally, the production environment undergoes the same smoke and Playwright end-to-end testing used for preview deployments. This verifies that the application is reachable and that important functionality still works after deployment.

5 Infrastructure and Security

Currently a single Terraform configuration is used for all deployments made. This is possible through the use of Terraform variables and HCP Terraform. Terraform variables allow us to specify different characteristics for each deployment such as different images, container names, etc. Since the Terraform configuration runs on GitHub Actions, we use HCP Terraform as a remote state manager for all deployed resources. Usually, when running Terraform without HCP Terraform, the state files are stored alongside the Terraform configuration. However, since we are now running on GitHub Actions and do not wish to have such files part of our git history, remote state management became an elegant solution.

The database containers use Docker volumes to ensure that data remain persistent and are not tied to the lifetime of a specific container instance. This allows us to redeploy the database container when the application is updated, without having to worry about data being lost. The Nginx-Proxy and Acme-Companion setup is responsible for routing traffic to the correct container and ensuring that TLS is set up correctly. This setup was done manually the first time we set up the VPS, but can be replicated with the provided Docker-compose file in the `/infra/vps` directory.

Trivy is a security scanner that elegantly integrates with CI/CD pipelines and is used by us to scan any Docker images that we have built before deployment. The scan will look for known vulnerabilities and leaked secrets, potentially avoiding catastrophic security incidents. We have also enabled GitHub's CodeQL scanner, which scans our code for potential vulnerabilities every time

code is pushed to any branch. Dependabot is enabled and will automatically scan for vulnerable dependencies and open a pull request if a newer non-vulnerable version is available. Alongside with our preview environments, this becomes an elegant pipeline since you almost instantly are able to test new dependency versions deployed live and merge them with production if you deem that they work well. All production secrets are stored in the GitHub repository secret manager and are automatically propagated to deployments on the VPS through Terraform where needed. This has also allowed us to enable GitHub secrets scanning, which scans our repository for any secret leaks. The main branch is also protected and requires developers to first create a pull request and that all tests pass without issues. Deployment workflows are also protected with safeguards for pull requests from forked repositories to avoid arbitrary access to our VPS or leakage of secrets.

6 Design Decisions and Trade-offs

Several design decisions were made to make deployments safer, easier to test, and more reliable. First, each PR gets its own preview environment instead of sharing one staging environment. This allows different changes to be tested independently and reduces the risk that one PR affects another. Docker images are tagged with the commit SHA, making it easier to identify the exact version of the code used to build each image. When a PR is merged, the workflow reuses the tested images in production when they are available rather than rebuilding them. This reduces the risk of deploying a version that has not been tested. The application and database are run in separate containers to make them easier to manage. Each preview has its own HCP Terraform workspace to keep its infrastructure separate.

However, all production and preview environments share one VPS, so a server failure could affect them all. The schema file for the database does not provide a full migration system for future changes. Deployments also depend on external services such as GitHub Actions and HCP Terraform. Finally, Nginx-Proxy and the initial server setup are managed separately from each environment's Terraform resources.

7 AI-Assisted Tools

The use of artificial intelligence tools has been to support our learning and understanding of the topics covered in this project. All artificial intelligence tools were used responsibly and according to the course criteria. GitHub Copilot was used as an efficiency tool to suggest limited amounts of code which was then validated and sometimes modified to fit our needs. Discussions with Large Language Models (LLMs) were sometimes held to improve understanding of certain topics and to troubleshoot problems. All decisions made in this project are our own, and any aid from artificial intelligence remained subject to our own judgment and validation.

8 Conclusion

To summarize this project, we have built a complete CI/CD pipeline around the original project with automated tests, preview deployments, and production deployments. All changes made to the code are tested in different stages, ranging from functionality, integration between systems, and finally end-to-end deployment tests. Additionally, we have enforced the safety of the implementation using code scanning tools such as Trivy, GitHub CodeQL, GitHub Secret Scanning, and automated Dependabot alerts and pull requests. This has been done with modern techniques such as Terraform, HCP Terraform, GitHub Actions, Trivy, and Docker. The final benefit of this implementation is the reduction in manual deployment work, systematic testing to reduce unexpected faults, and consistent preview environments where deployments can be easily tested.